ĐỒ ÁN BẢO MẬT PHẦN MỀM

LearnSpace

Hệ thống quản lý học tập và học liệu có kiểm soát truy cập

Trình bày kiến trúc, luồng dữ liệu và các cơ chế bảo vệ người dùng, API, file và cơ sở dữ liệu.

L

NGUYỄN HẢI PHONG

React · ASP.NET Core
MongoDB Atlas

2026

Bài toán của đề tài

Một hệ thống dùng chung nhưng mỗi người chỉ được thấy và làm đúng phần việc của mình.

Quản lý học tập

Lớp, bài học, bài tập, lịch, đăng ký, điểm và học phí.

Quản lý nội dung

Học liệu, phiên bản, kiểm duyệt, ngân hàng câu hỏi và quiz.

Bảo vệ dữ liệu

Ngăn xem sai lớp, sai khoa, lộ mật khẩu, đáp án hoặc file riêng tư.

Câu hỏi trung tâm: làm sao hệ thống vẫn dễ sử dụng nhưng không tin dữ liệu và quyền do trình duyệt tự khai báo?

LearnSpace giải quyết bài toán như thế nào?

Chức năng nghiệp vụ và bảo mật được thiết kế cùng nhau, không tách thành hai phần độc lập.

1

Đăng nhập

Xác minh tài khoản và tạo cookie phiên.

2

Kiểm tra quyền

Role + permission quyết định chức năng.

3

Kiểm tra phạm vi

Lớp, khoa, người sở hữu và trạng thái.

4

Xử lý dữ liệu

Typed query, GridFS và update có điều kiện.

5

Trả kết quả

DTO an toàn và ghi audit khi cần.

Bốn vai trò và phạm vi dữ liệu

Role cho biết người dùng thuộc nhóm nào; phạm vi cho biết họ được truy cập bản ghi nào.

Admin

Quản trị toàn hệ thống

Người dùng, quyền, khoa, ngành, học kỳ.

Staff

Cán bộ theo khoa

Sinh viên, học liệu, hỗ trợ và báo cáo trong khoa.

Teacher

Giảng viên theo lớp/môn

Bài học, bài tập, câu hỏi và quiz được phân công.

Student

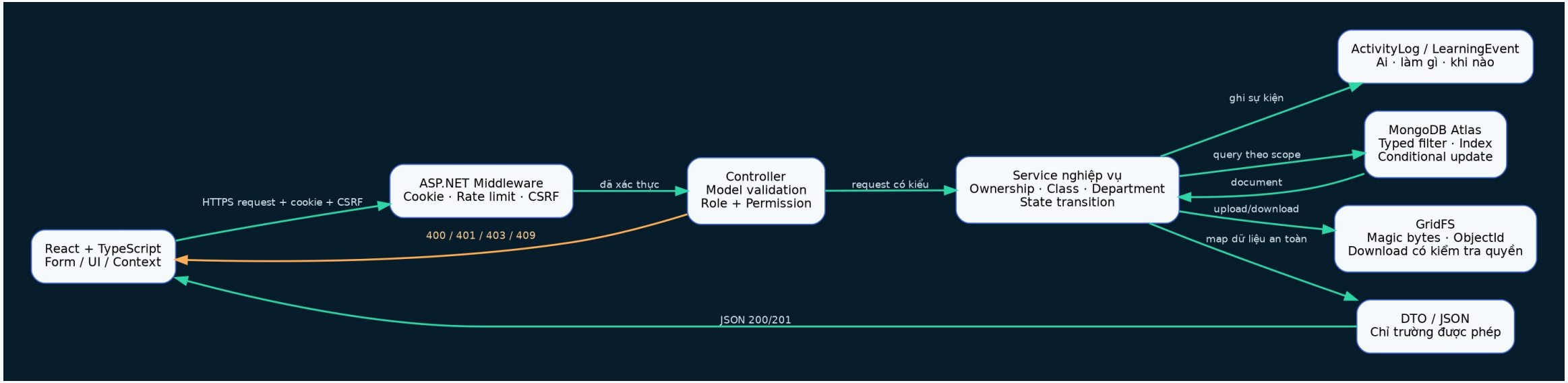
Người học chính chủ

Lớp đã ghi danh, bài nộp, điểm và ticket của mình.

Một Teacher có quyền “quản lý bài tập” vẫn không được sửa bài tập của lớp không được phân công.

Kiến trúc tổng thể

Mọi truy cập dữ liệu đều đi qua backend; React không kết nối trực tiếp tới MongoDB.



Các công nghệ chính và lý do sử dụng

Mỗi công nghệ đảm nhiệm một vai trò cụ thể trong hệ thống.

React + TypeScript

Giao diện SPA; TypeScript phát hiện nhầm kiểu khi build.

ASP.NET Core

API, middleware, validation, authentication và authorization.

MongoDB Atlas

Lưu document, index, TLS và vận hành database trên cloud.

GridFS

Lưu file ngoài webroot và tải file thông qua API có kiểm tra quyền.

BCrypt + Cookie

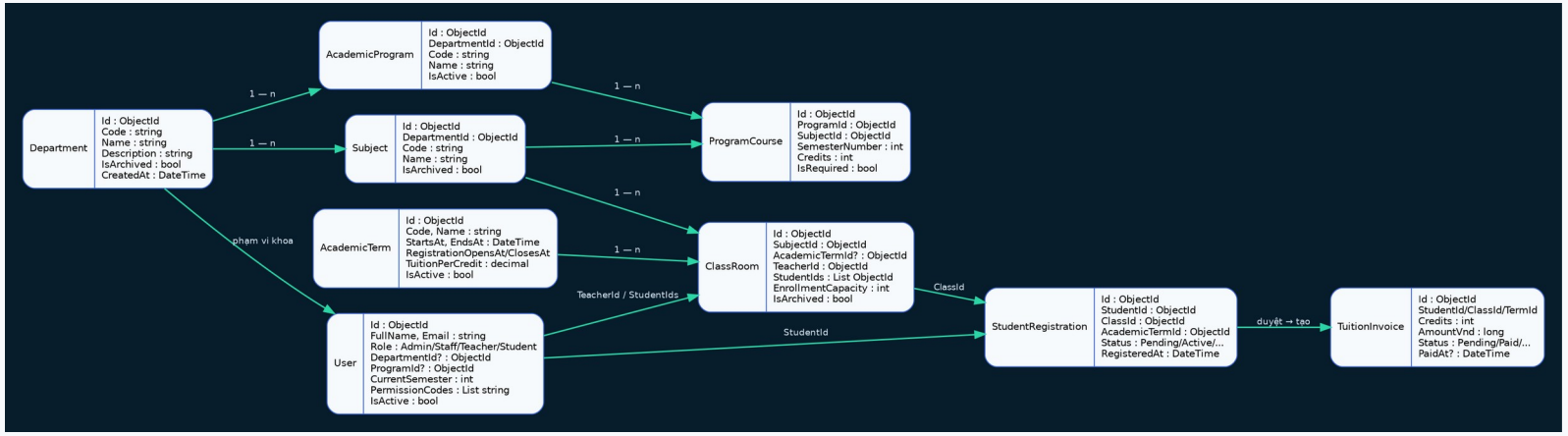
Băm mật khẩu; cookie HttpOnly duy trì phiên đăng nhập.

Docker + Render

Đóng gói và chạy cùng một cấu hình khi triển khai.

Mô hình dữ liệu cốt lõi

MongoDB liên kết các collection bằng ObjectId; dữ liệu con có thể được nhúng thành mảng.



Liên kết bằng ObjectId

Ví dụ Subject.DepartmentId trỏ tới Department.Id.

Dữ liệu nhúng

Messages[], Versions[] và Question snapshots nằm trong document cha.

Không phải SQL foreign key

Service chịu trách nhiệm kiểm tra tồn tại và phạm vi liên kết.

Một request được kiểm tra ở ba lớp

Chỉ kiểm tra role ở controller là chưa đủ để bảo vệ dữ liệu cụ thể.

1. Authentication	Người dùng đã đăng nhập chưa? Cookie còn hợp lệ không?
2. Authorization	Role và permission có cho phép chức năng này không?
3. Data scope	Bản ghi có thuộc lớp, khoa hoặc chính người dùng đó không?

Kết quả: chưa đăng nhập → 401 · thiếu quyền → 403 · dữ liệu sai → 400 · xung đột trạng thái → 409

Luồng đăng nhập

Mật khẩu không được lưu dạng rõ; kết quả lỗi không tiết lộ email có tồn tại hay không.

CODE THỰC TẾ ĐƯỢC RÚT GỌN

```
var user = await FindByEmail(email);
if (user is null || !user.IsActive ||
    user.LockoutUntil > DateTime.UtcNow)
    return Failed();

var valid = BCrypt.Verify(password,
                          user.PasswordHash);
await UpdateFailedAttemptsAtomically(user, valid);
```

- Tìm email đã chuẩn hóa trong MongoDB.
- Kiểm tra tài khoản hoạt động và thời gian khóa.
- BCrypt so sánh mật khẩu với hash.
- Sai nhiều lần sẽ tăng bộ đếm bằng update nguyên tử.
- Thông báo chung: “Email hoặc mật khẩu không đúng”.

Cookie phiên đăng nhập được bảo vệ như thế nào?

Cookie lưu định danh đã ký; JavaScript không đọc được và server kiểm tra lại tài khoản mỗi request.

HttpOnly

JavaScript trong trang không đọc được cookie.

Secure + SameSite

Production chỉ gửi qua HTTPS; hạn chế gửi từ trang khác.

KIỂM TRA LẠI COOKIE Ở MỖI REQUEST

```
var version = principal.FindFirstValue(
    AuthClaims.PasswordVersion);

if (!user.IsActive || user.Role != roleClaim ||
    version != PasswordVersionFor(user))
{
    context.RejectPrincipal();
    await context.HttpContext.SignOutAsync();
}
```

Đổi mật khẩu, đổi role hoặc khóa tài khoản → cookie cũ mất hiệu lực.

Mật khẩu và quên mật khẩu bằng OTP

OTP chỉ có giá trị khi email đã được máy chủ SMTP chấp nhận gửi.

BCrypt

Hash có salt, không lưu mật khẩu rõ.

Lịch sử

Giữ 5 hash gần nhất để chống dùng lại.

OTP

6 số, hiệu lực 10 phút, dùng một lần.

Giới hạn

Cooldown 60 giây, tối đa 5 email/giờ.

Phiên cũ

Reset thành công vô hiệu mọi cookie cũ.

TLS email

Không bỏ qua kiểm tra chứng chỉ SMTP.

Bảo vệ dữ liệu cá nhân và chống IDOR

IDOR xảy ra khi người dùng đổi ID trên URL để truy cập bản ghi của người khác.

KIỂM TRA PHẠM VI DỮ LIỆU

```
var classroom = await classes.GetAsync(classId);

if (user.Role == Roles.Teacher)
    return classroom.TeacherId == user.Id;

if (user.Role == Roles.Student)
    return classroom.StudentIds.Contains(user.Id);

if (user.Role == Roles.Staff)
    return subject.DepartmentId == user.DepartmentId;
```

Teacher A ≠ TeacherId → 403

Student không thuộc roster → 403

Staff khác khoa → không trả dữ liệu

Ba rủi ro web thường gặp

Mỗi rủi ro cần một cơ chế phòng vệ riêng.

CSRF

Nguy cơ
Trang ngoài lợi dụng cookie đang đăng nhập để gửi lệnh.

Cách bảo vệ
Antiforgery token cho request thay đổi dữ liệu.

XSS

Nguy cơ
Nội dung người dùng chứa script chạy trong trình duyệt.

Cách bảo vệ
React escape chuỗi; không render HTML tùy ý.

NoSQL Injection

Nguy cơ
Dữ liệu nhập bị biến thành toán tử truy vấn MongoDB.

Cách bảo vệ
Model có kiểu và typed filter ở backend.

CSRF được chặn ở đâu?

Trình duyệt phải gửi token do chính ứng dụng cấp; chỉ có cookie là chưa đủ.

CONTROLLER + REACT

```
// Controller
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(...)

// React gửi token trong header
RequestVerificationToken: csrfToken
```

- GET chỉ đọc dữ liệu nên không thay đổi trạng thái.
- POST, PUT và DELETE từ trình duyệt phải có token.
- Token sai hoặc thiếu → 400 và database không đổi.
- Cookie vẫn đặt HttpOnly, Secure và SameSite.

XSS và NoSQL Injection được chặn như thế nào?

Frontend chịu trách nhiệm hiển thị an toàn; backend chịu trách nhiệm truy vấn an toàn.

XSS

```
// React tự escape chuỗi
<p>{ticket.message}</p>

// Không dùng:
dangerouslySetInnerHTML={{ __html: input }}
```

NOSQL INJECTION

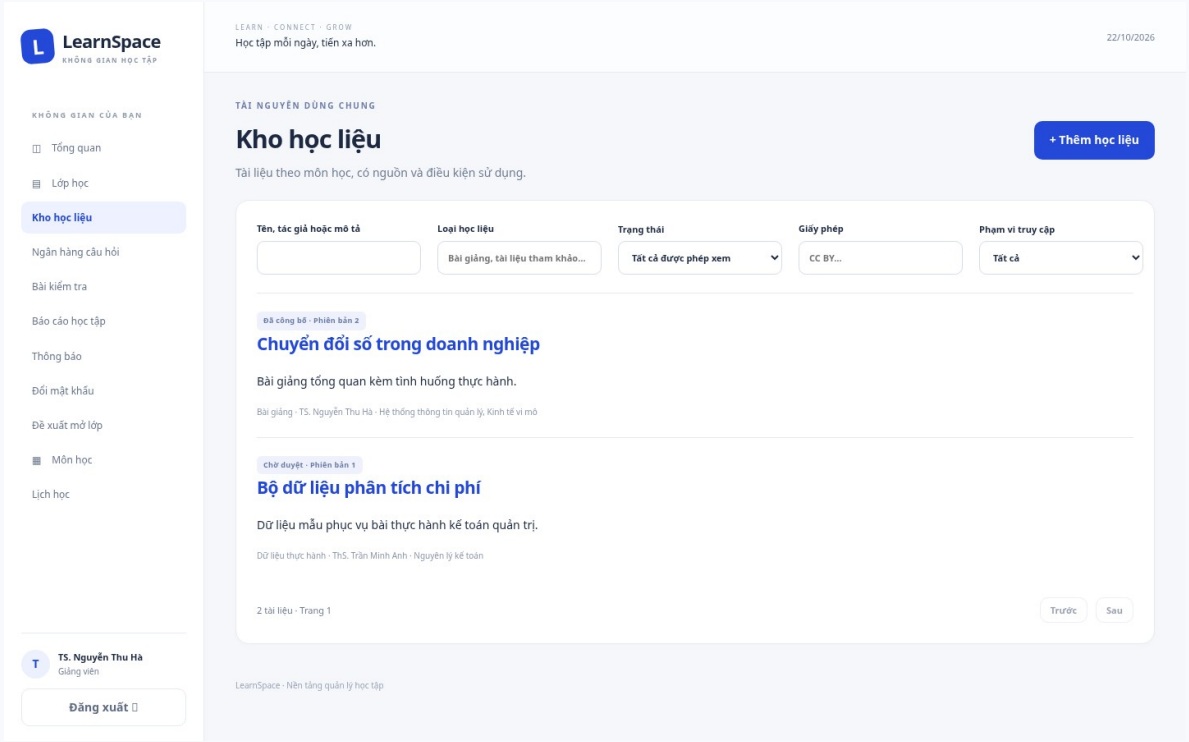
```
// Typed filter của MongoDB Driver
var user = await Users.Find(
    x => x.Email == normalizedEmail
).FirstOrDefaultAsync();

// Client không được gửi raw BSON filter.
```

Payload như <script> chỉ hiện thành chữ; email {"$ne": null} bị validation từ chối.

Bảo vệ chức năng tải file

Không tin tên file, phần mở rộng hoặc MIME do trình duyệt gửi lên.



- Giới hạn 25 MB/file và tối đa 5 file/lượt.
- Allowlist phần mở rộng được chấp nhận.
- Kiểm tra magic bytes để xác nhận nội dung thật.
- Lưu file bằng ObjectId trong GridFS, ngoài webroot.
- Download kiểm tra owner, lớp và quyền hiện tại.
- File có nội dung thực thi nhưng đổi tên .pdf → 400.

Nội dung phải qua kiểm duyệt

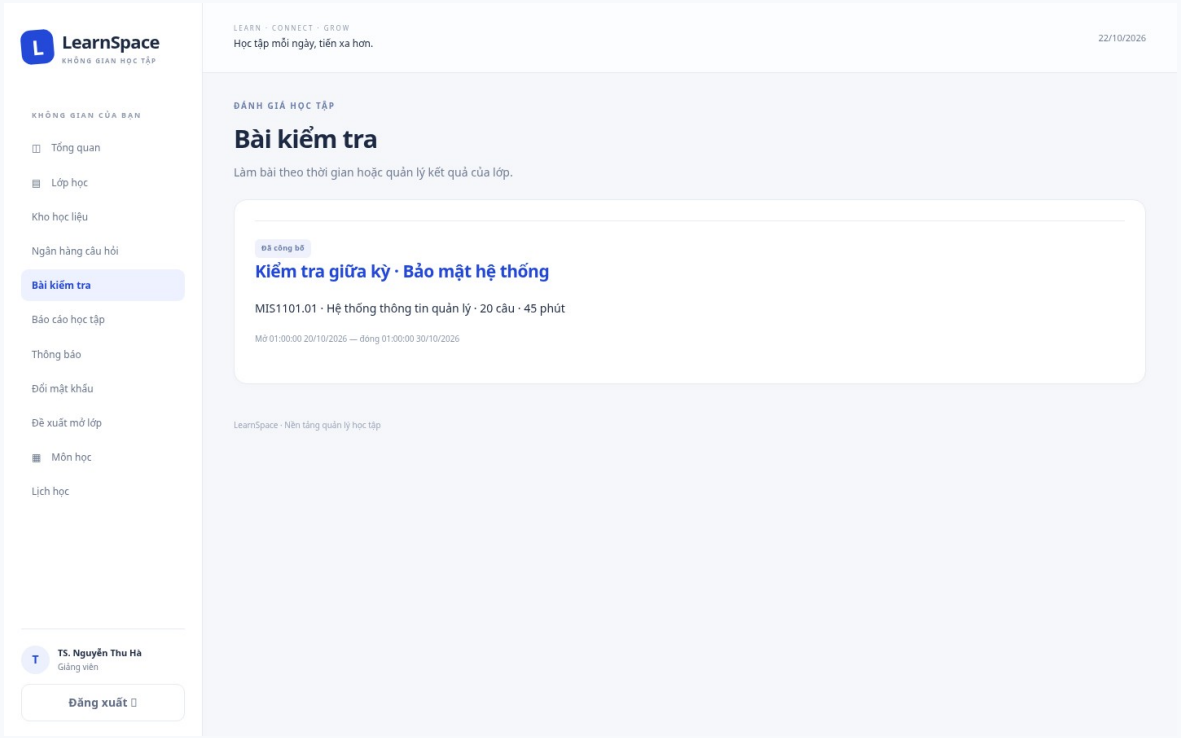
Tác giả tạo nội dung; người khác có thẩm quyền mới được duyệt và công bố.



Quy tắc quan trọng: tác giả không tự duyệt · reviewer đúng khoa · revision ngăn ghi đè thay đổi mới.

Bài kiểm tra được bảo vệ như thế nào?

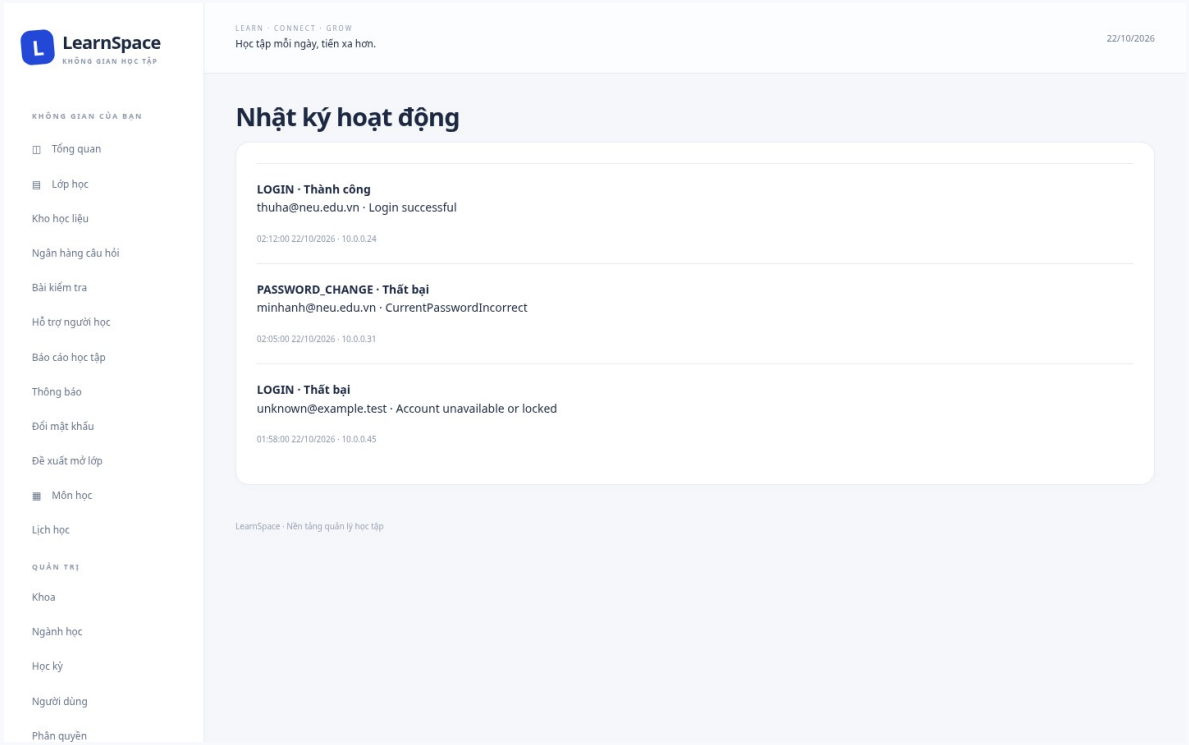
Đáp án và việc chấm điểm chỉ được xử lý ở server.



- Quiz chỉ dùng câu hỏi đã Published và đúng môn.
- Student phải thuộc lớp và quiz đang trong thời gian mở.
- API không trả CorrectOption trước khi nộp.
- QuizAttempt lưu snapshot câu hỏi để giữ lịch sử.
- Server kiểm tra số đáp án và chỉ chấm một lần.
- Kết quả cũ không thay đổi khi câu hỏi gốc được sửa.

Nhật ký và xử lý lỗi an toàn

Log đủ để truy vết nhưng không được trở thành nơi làm lộ secret.



ActivityLog

Login, đổi mật khẩu, IP, user agent và kết quả.

LearningEvent

Quiz, học liệu, support và hoạt động học tập.

Lỗi production

Người dùng nhận thông báo chung + request ID; chi tiết ở log server.

Giữ hệ thống ổn định và có thể phục hồi

Bảo mật còn bao gồm khả năng chống lạm dụng, phát hiện lỗi và khôi phục dữ liệu.

Rate limit

Giới hạn login, OTP và AI; vượt ngưỡng trả 429.

Health check

/live kiểm tra tiến trình; /ready kiểm tra MongoDB.

Index + pagination

Giảm truy vấn quét toàn bộ collection.

Backup / restore

Khôi phục thử vào database riêng trước khi tin bản backup.

Dependency scan

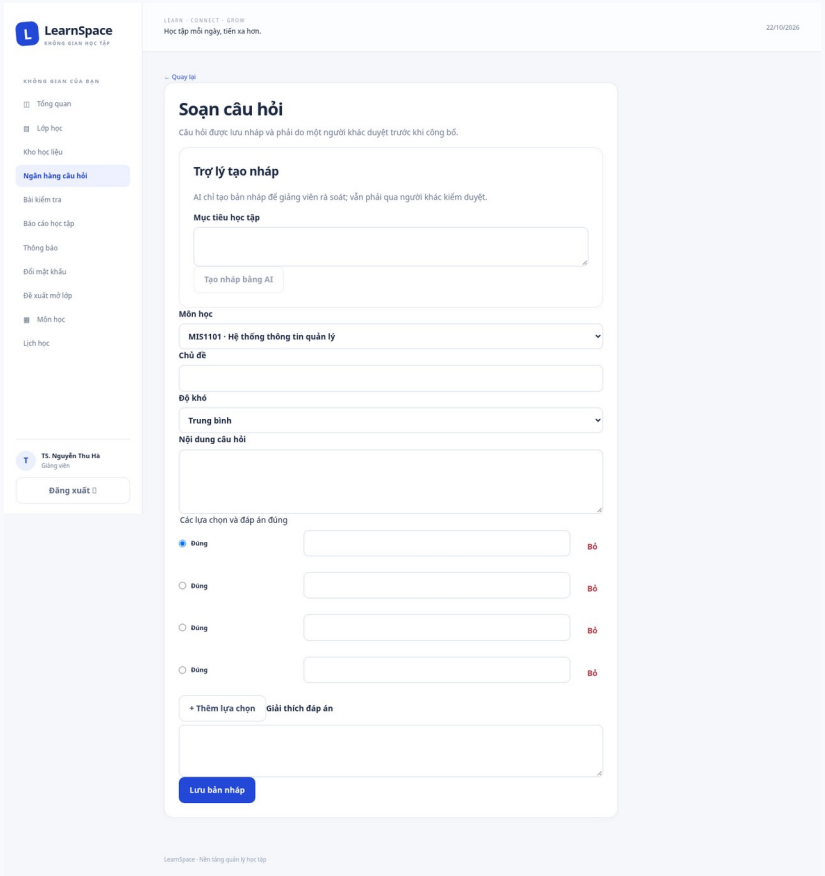
Quét npm và NuGet để tìm thư viện có CVE.

Secret ngoài source

URI, SMTP, VNPAY và AI key dùng biến môi trường.

Tích hợp AI theo hướng có kiểm soát

AI hỗ trợ soạn câu hỏi nhưng không được tự xuất bản nội dung.



- API key chỉ nằm ở backend.
- Structured Outputs buộc phản hồi đúng JSON Schema.
- Kết quả luôn ở trạng thái Draft và có cờ AI Assisted.
- Teacher vẫn phải sửa, gửi duyệt và publish.
- Student không có quyền gọi AI draft.
- AI lỗi hoặc chưa cấu hình → không tạo dữ liệu dở dang.

Kiểm thử chứng minh điều gì?

Test chạy với database riêng và kiểm tra cả response lẫn dữ liệu sau request.

64
nhóm regression

20/20
luồng React

96
request đồng thời

3/3
document restore

- ✓ Sai role gọi API → 403
- ✓ POST thiếu CSRF → database không đổi
- ✓ OTP sai/hết hạn/dùng lại → từ chối
- ✓ Đổi ObjectId → không thấy dữ liệu người khác
- ✓ File giả PDF → 400
- ✓ Lỗi production → không lộ stack trace

Kịch bản demo trực tiếp

Demo theo một câu chuyện ngắn để người xem thấy quyền và dữ liệu thay đổi theo vai trò.

1

Admin

Xem và gán quyền

2

Staff

Chỉ thấy sinh viên trong khoa

3

Teacher

Mở lớp, học liệu và quiz

4

Student

Học, nộp bài và xem báo cáo

5

Admin

Xem ActivityLog

Trong mỗi bước: chỉ ra chức năng được phép và thử một thao tác ngoài phạm vi để thấy server từ chối.

KẾT LUẬN

Bảo mật là một phần của luồng nghiệp vụ

LearnSpace không chỉ ẩn nút trên giao diện. Mỗi request được xác thực, phân quyền và kiểm tra phạm vi dữ liệu ở server.

Đúng người

Đúng chức năng

Đúng dữ liệu

Có thể truy vết

Cookie · BCrypt · Role/Permission/Scope · CSRF · GridFS · Audit · Backup

Q&A

Cảm ơn thầy cô
và các bạn

github.com/nguyenhaiphong0711-bit/BaoMat